Machine Learning Workshop

Retail Customer Behavioural Analysis

PROJECT REPORT

Practical Workshop : E-commerce Gift Retail

Prepared by MARYEM KBAYER

Table des matières

Executive Summary **3**

1 Understanding the Raw Dataset **4**
1.1 Immediately Suspicious Variables . . . 4

2 Data Preprocessing (`preprocessing.py`) **6**
2.1 Parsing Registration Date . . . 6
2.2 Parsing IP Address . . . 6
2.3 Dropping Useless, Leaky, and Redundant Columns . . . 7
2.4 Cleaning Sentinel Values . . . 7
2.5 Encoding Categorical Variables . . . 8
2.5.1 Ordinal Encoding — Variables with a Natural Order . . . 8
2.5.2 One-Hot Encoding — Unordered Variables . . . 8
2.5.3 Target Encoding — Country (Smoothed, Leak-Free) . . . 9
2.6 Train / Test Split . . . 9
2.7 Missing Value Imputation . . . 9
2.8 Feature Engineering . . . 10
2.9 Logarithmic Transformations . . . 10
2.10 Normalisation (StandardScaler) . . . 10
2.11 PCA (Principal Component Analysis) . . . 10

3 Model Training (`train_model.py`) **13**
3.1 Imbalance Management : Weighting and Threshold Optimisation . . . 13
3.2 Comparison of Three Models on Two Feature Sets . . . 13
3.3 Hyperparameter Tuning : GridSearchCV vs Optuna . . . 14
3.3.1 Method A — GridSearchCV . . . 14
3.3.2 Method B — Optuna (Bayesian Optimisation) . . . 15
3.4 Probability Calibration . . . 15
3.5 Decision Threshold Optimisation . . . 16
3.6 Final Evaluation on the Test Set . . . 16
3.7 Feature Importance . . . 17
3.8 Surrogate Decision Tree . . . 17

4 Customer Clustering (`clustering.py`) **19**
4.1 Finding the Right Number of Clusters . . . 19
4.1.1 Method 1 — Elbow Method . . . 19
4.1.2 Method 2 — Silhouette Score . . . 19
4.2 Cluster Profiles . . . 20
4.2.1 Business Interpretation and Marketing Recommendations . . . 21

5 Spend Prediction Regression (`regression.py`) **22**
5.1 Target Variable and Outlier Handling . . . 22
5.2 Model Comparison (5-Fold Cross-Validation) . . . 22
5.3 Hyperparameter Tuning : GridSearchCV vs Optuna . . . 22
5.4 Final Evaluation on the Test Set . . . 23
5.5 Residual Analysis by Spend Percentile . . . 23

6 Model Monitoring (`monitoring.py`) **25**
6.1 What Was Monitored . . . 25
6.2 Drift Detection Results . . . 25
6.3 Performance Monitoring Results . . . 25

7 Production Prediction Pipeline (`predict.py`) 26
7.1 Artifacts Loaded at Startup . . . 26
7.2 The Inference Pipeline (Step by Step) . . . 26
7.3 Output Risk Levels . . . 26
7.4 Live Demonstration . . . 27

8 Deployment and Recommendations 28
8.1 Deployment : Architecture and Web Interface . . . 28
8.1.1 Backend Logic and Inference . . . 28
8.1.2 User Interface and Output . . . 28
8.2 Strategic Recommendations . . . 28
8.2.1 Priority Short-Term Actions (0–3 months) . . . 29
8.2.2 Medium-Term Technical Improvements (3–12 months) . . . 29
8.2.3 Limitations and Usage Precautions . . . 29

9 Generated Files and Artifacts 30
9.1 Saved Model Artifacts . . . 30
9.2 Saved Datasets . . . 30

Conclusions 31

Executive Summary

This report details every step of a machine learning project designed to predict which customers of a retailer are likely to churn. Starting from a raw dataset of 4,372 customers, we clean and transform it, train several models, select the best one, validate it, and deploy it for real-time predictions. The final model — a *Gradient Boosting* classifier tuned with Optuna and calibrated for reliable probabilities — achieves excellent performance on previously unseen customers :

ROC-AUC	Churners Detected	F1-Score	Churners Missed
0.9817	**93.47%**	**0.8962**	**19 / 291**

In simple terms : out of 100 true churners, the model correctly identifies 93. It only misses 7. This allows the company to proactively contact almost all at-risk customers before they leave.

1 Understanding the Raw Dataset

Starting point : a CSV file containing 4,372 rows (customers) and 52 columns. Each row corresponds to one customer. The columns represent everything known about them : purchase frequency, amount spent, location, tenure, age, preferences, etc. The most important column is `Churn` — the target variable to predict. A value of `1` means the customer has churned (lost). A value of `0` means they are still active (*Loyal*).

```
Raw shape : (4372, 52)
```

Listing 1 – Terminal output — initial load

Résultat

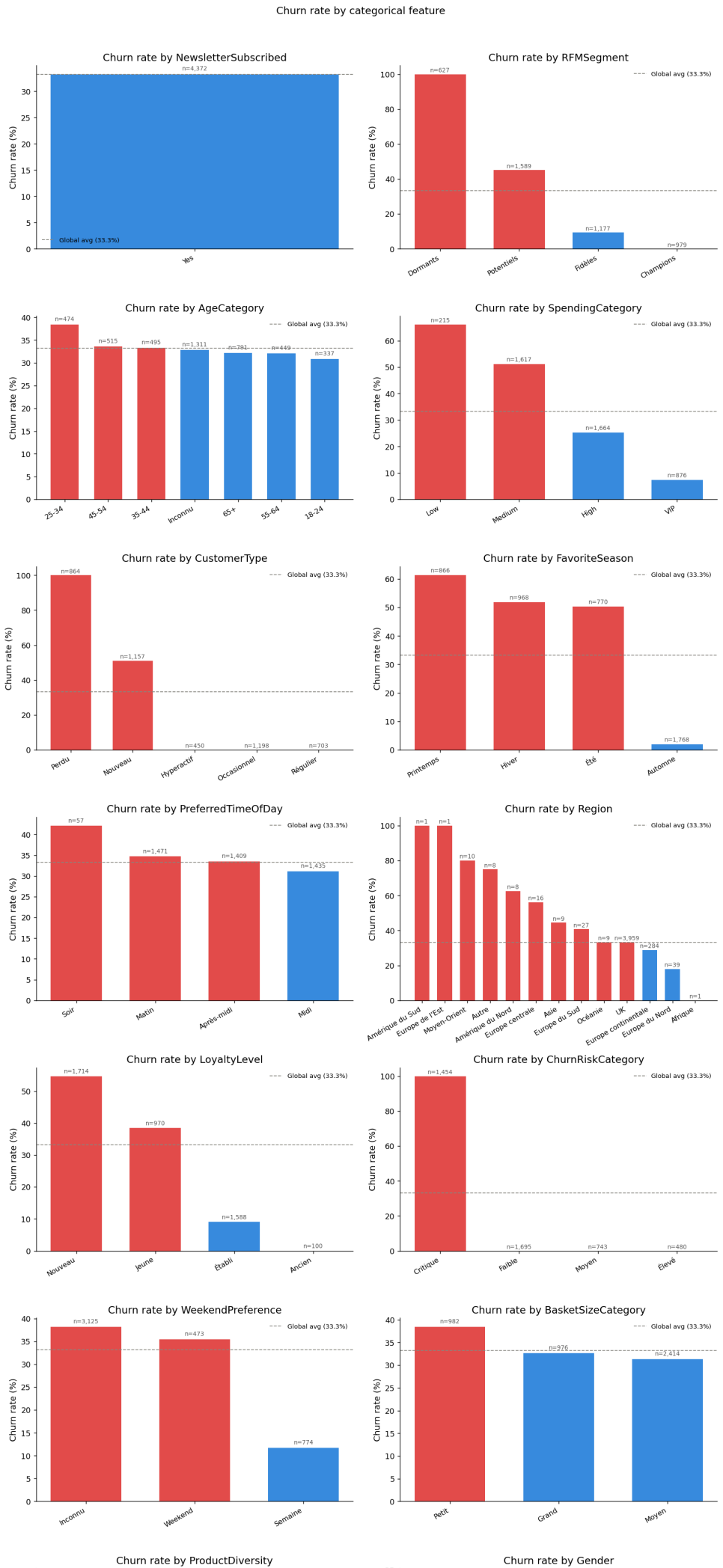
4,372 customers × 52 columns loaded successfully. Churn rate : **33.26%** — approximately 1 in 3 customers has churned. Moderately imbalanced problem.

1.1 Immediately Suspicious Variables

Before anything else, we identified columns that directly reveal whether a customer has churned, even though they should not. These are **leaky variables** (*data leakage*) : they cheat by giving the answer to the model during training.

- **Recency** : correlation of 0.86 with Churn — it is literally the definition of churn.
- **CustomerType** : contains the value *Lost*, which directly encodes the label.
- **ChurnRiskCategory, RFMSegment, LoyaltyLevel** : all derived from churn status.
- **FirstPurchaseDaysAgo** : constant value of 374 for all customers. Zero predictive signal.
- **SatisfactionScore** : p-value = 0.365, correlation = +0.014. Statistically useless.

The following figure illustrates the churn distribution across customer categories. Some categories contain mostly churned customers, confirming the presence of information leakage.


Churn rate by categorical feature
Churn rate by NewsletterSubscribed
n=4,372
Global avg (33.3%)
Yes
Churn rate by RFMSegment
n=627
n=1,589
n=1,177
n=979
Dormants
Potentiels
Fidèles
Champions
Churn rate by AgeCategory
n=474
n=515
n=495
n=1,311
n=391
n=449
n=337
25-34
45-54
35-44
Inconnu
65+
55-64
18-24
Churn rate by SpendingCategory
n=215
n=1,617
n=1,664
n=876
Low
Medium
High
VIP
Churn rate by CustomerType
n=864
n=1,157
n=450
n=1,198
n=703
Perdu
Nouveau
Hyperactif
Occasionnel
Régulier
Churn rate by FavoriteSeason
n=866
n=968
n=770
n=1,768
Printemps
Hiver
Été
Automne
Churn rate by PreferredTimeOfDay
n=57
n=1,471
n=1,409
n=1,435
Soir
Matin
Après-midi
Midi
Churn rate by Region
n=1
n=1
n=10
n=8
n=8
n=16
n=9
n=27
n=9
n=3,959
n=284
n=39
n=1
Amérique du Sud
Europe de l'Est
Moyen-Orient
Autre
Amérique du Nord
Europe centrale
Asie
Europe du Sud
Océanie
UK
Europe continentale
Europe du Nord
Afrique
Churn rate by LoyaltyLevel
n=1,714
n=970
n=1,588
n=100
Nouveau
Jeune
Établi
Ancien
Churn rate by ChurnRiskCategory
n=1,454
n=1,695
n=743
n=480
Critique
Faible
Moyen
Élevé
Churn rate by WeekendPreference
n=3,125
n=473
n=774
Inconnu
Weekend
Semaine
Churn rate by BasketSizeCategory
n=982
n=976
n=2,414
Petit
Grand
Moyen
Churn rate by ProductDiversity
Churn rate by Gender
Churn rate (%)
Global avg (33.3%)

2 Data Preprocessing (preprocessing.py)

Preprocessing is the stage where we clean the raw data and transform it into a format usable by a machine learning model. Every decision is made carefully to avoid any data leakage.

Golden Rule : fit everything (medians, encoders, scalers) *only* on the training data. Apply to the test set without ever letting it influence the fitting process.

2.1 Parsing Registration Date

The `RegistrationDate` column contained dates in three different formats : UK format (`12/09/2009`), ISO (`2010-10-04`), and US (`10/18/2010`). We used `pandas.to_datetime` with `format="mixed"` and extracted three new numeric columns (RegYear is omitted as it is redundant with CustomerTenureDays) :

— **RegMonth** — month (1 to 12)

— **RegDay** — day of the month (1 to 31)

— **RegWeekday** — day of the week (0=Monday, 6=Sunday)

```
Extract: RegMonth, RegDay, RegWeekday (RegYear ignored - redundant)
RegistrationDate dropped after extraction
```

Résultat

3 clean numeric variables extracted from a heterogeneous text column.

Bug corrigé

`dayfirst=True` caused a `UserWarning` for dates in US format (non-existent month 18). Fixed by switching to `format="mixed"` with `dayfirst=False`, letting `dateutil` handle all formats automatically.

2.2 Parsing IP Address

The `LastLoginIP` column contained raw strings (e.g., `85.244.30.10`). Two new variables were created :

— **IsPrivateIP** : 1 if the IP belongs to the RFC1918 space (private), 0 otherwise.

— **IPClass** : 1 = Class A (1–126), 2 = Class B (128–191), 3 = Class C (192–223), 0 = other.

```
IsPrivateIP : {0: 4046, 1: 326}
IPClass     : {0: 595, 1: 2175, 2: 1045, 3: 557}
LastLoginIP dropped after extraction
```

Résultat

2 binary/ordinal variables created from a raw IP string.

2.3 Dropping Useless, Leaky, and Redundant Columns

Column	Category	Reason for Dropping
Recency	LEAKY	0.859 correlation with Churn — defines churn
CustomerType	LEAKY	Contains *Lost* — directly encodes the label
ChurnRiskCategory	LEAKY	Derived from churn status
RFMSegment	LEAKY	Dormant = churned. Pure leak
LoyaltyLevel	LEAKY	Directly encodes retention
CustomerID	IDENTIFIER	Random identifier, no signal
NewsletterSubscribed	QUASI-CONSTANT	Identical value for almost everyone
FirstPurchaseDaysAgo	CONSTANT	Value of 374 for everyone — statistical artifact
SatisfactionScore	NON-SIGNIFICANT	$p = 0.365$, corr $= +0.014$
AvgLinesPerInvoice	MULTICOLLINEAR	$r = 0.96$ with AvgProductsPerTransaction
TotalQuantity	MULTICOLLINEAR	$r = 0.92$ with MonetaryTotal
MonetaryStd / MonetaryMin	MULTICOLLINEAR	Entangled cluster with MonetaryMax
MinQuantity / MaxQuantity	MULTICOLLINEAR	Entangled with MonetaryMin/Max
TotalTransactions	MULTICOLLINEAR	$r = 0.88$ with UniqueInvoices
RegYear	REDUNDANT	Inversely correlated with CustomerTenureDays
PreferredMonth	TEMPORAL LEAK	Encodes recent activity (seasonal bias)

```
Shape after drops : (4372, 35)
```

Résultat

Dataset reduced from 52 to 35 columns. All remaining columns are clean, leak-free, and carry signal.

2.4 Cleaning Sentinel Values

Some columns used "magic numbers" to represent missing data :

— **SupportTicketsCount** : values -1 and 999 replaced by `NaN`.

— **IsPrivateIP** : value -1 (unparsable IP) replaced by `NaN`.

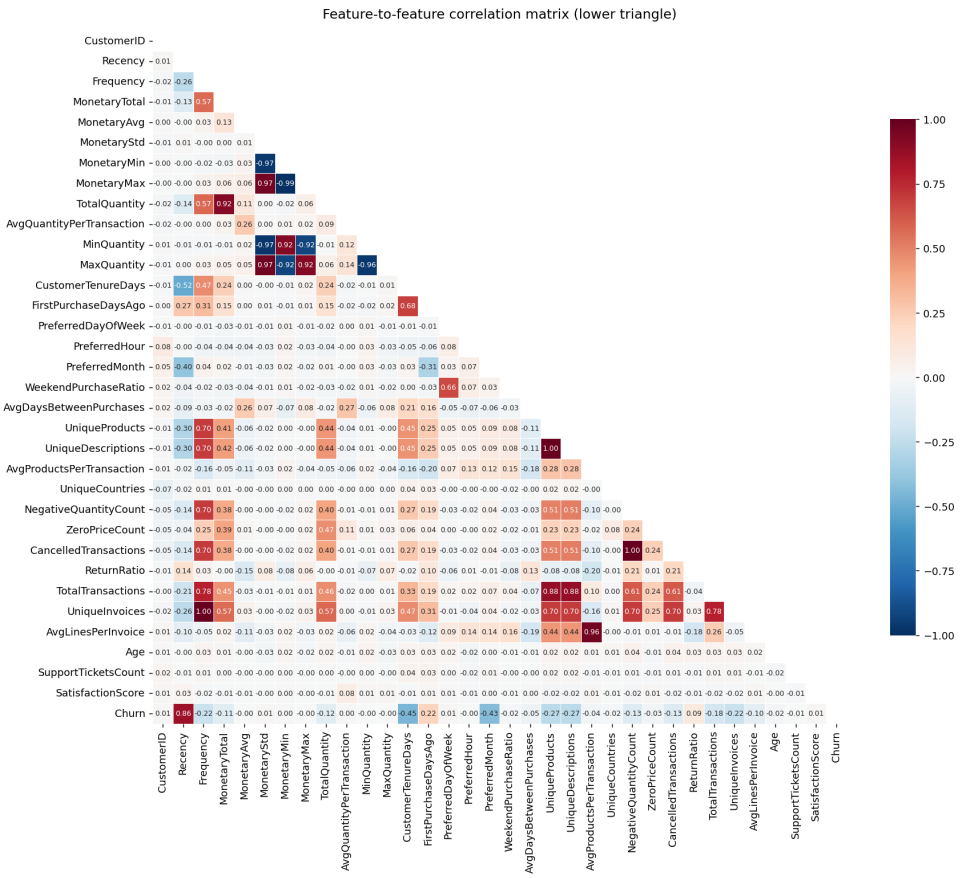


FIGURE 2 – Correlation heatmap of numerical features.

Résultat

Sentinel values replaced. The model will no longer confuse −1 or 999 with real values.

2.5 Encoding Categorical Variables

2.5.1 Ordinal Encoding — Variables with a Natural Order

— **SpendingCategory** : Low=0, Medium=1, High=2, VIP=3
— **AgeCategory** : 18-24=0, 25-34=1, 35-44=2, 45-54=3, 55-64=4, 65+=5
— **BasketSizeCategory** : Small=0, Medium=1, Large=2, Unknown=3
— **PreferredTimeOfDay** : Night=0, Morning=1, Noon=2, Afternoon=3, Evening=4

2.5.2 One-Hot Encoding — Unordered Variables

```
OHE columns: ['FavoriteSeason', 'Region', 'WeekendPreference',
              'ProductDiversity', 'Gender', 'AccountStatus']
Shape after OHE (before target encoding): (4372, 53)
```

2.5.3 Target Encoding — Country (Smoothed, Leak-Free)

The `Country` variable was encoded using the average churn rate per country, with smoothing ($k = 10$) to avoid memorizing rare countries.

$$\text{smoothed_value} = \frac{n \cdot \bar{y}_{\text{country}} + k \cdot \bar{y}_{\text{global}}}{n + k} \qquad (k = 10)$$

Country	Smoothed Rate	n (train)
Unspecified	0.523	4
Canada	0.523	4
Greece	0.444	2
Japan	0.431	7
Israel	0.410	3

Résultat

Country encoded into a single numeric column (smoothed churn rate). Unknown countries in production : default value **0.333** (global rate).

2.6 Train / Test Split

The dataset was split into 80% training (3,497 customers) and 20% test (875 customers) **before** any imputation, scaling, or feature engineering.

```
X_train : (3497, 52) | X_test : (875, 52)
Churn rate -- train: 33.26% | test: 33.26%
```

Résultat

Stratified split : same churn rate in both sets. No data leakage.

2.7 Missing Value Imputation

Column	Strategy	Value	Note
Age	Median	49.0	+ flag `Age_IsMissing`
AgeCategory	Median	3.0 (45–54)	Handles *Unknown*s
AvgDaysBetweenPurchases	Fill 0	0	Correct for single purchase
SupportTicketsCount	Median	2.0	After replacing sentinels
RegMonth / RegDay / RegWeekday	Median	Train medians	Unparsed dates
IsPrivateIP	Mode	0 (public)	Unparsable IPs

Résultat

All missing values filled. No `NaN`s remain after this step.

2.8 Feature Engineering

4 new columns were created from existing columns :

Variable	Formula	What It Measures
AvgBasketValue	MonetaryTotal / (Frequency + 1)	Average spend per visit
EngagementScore	Frequency / (CustomerTenureDays + 1)	Frequency relative to tenure
DisengagementScore	ReturnRatio + (CancelledTransactions / 10)	Negative behaviour signal
RevenueIndex	MonetaryTotal / (mean_global + 1)	Spend vs global average

```
RevenueIndex global_mean (train) : 1908.1900
Shape after feature engineering -- train: (3497, 57) | test: (875, 57)
```

2.9 Logarithmic Transformations

The columns `MonetaryTotal`, `Frequency`, and `AvgBasketValue` had highly skewed distributions. We applied the $\log(1 + x)$ transformation to compress extreme values.

```
log1p applied to: MonetaryTotal, Frequency, AvgBasketValue
```

2.10 Normalisation (StandardScaler)

```
Scaled 54 numeric columns
Final shape -- train: (3497, 57) | test: (875, 57)
Scaler saved: models/scaler.pkl
```

Résultat

Scaler fitted on the training set only. Applied to the test set using training statistics. No leakage.

2.11 PCA (Principal Component Analysis)

After normalisation, we have 57 variables. To test if dimensionality reduction could improve performance, we applied a PCA with a 95% retained variance threshold. The PCA was fitted only on `X_train` and then applied to `X_test` — no information from the test set influenced the transformation.

```
Components selected : 39  (from 54 numeric features)
Variance retained   : 95.20%
X_train_pca : (3497, 39)
X_test_pca  : (875, 39)
```

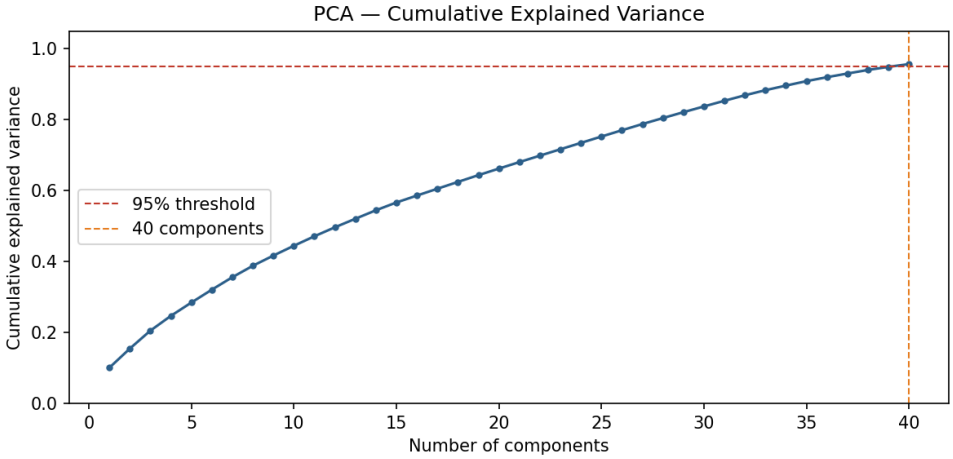


FIGURE 3 – Cumulative variance curve of the PCA. The 95% threshold is reached at 39 components, explaining 95.20% of the total variance of the numeric variables.

The gradual curve with no sharp elbow confirms that the information in this dataset is distributed across many variables — there is no single dominant signal. This means that overly aggressive compression would lose important information. Therefore, we kept 39 components as the optimal tradeoff between dimensionality reduction and information preservation.

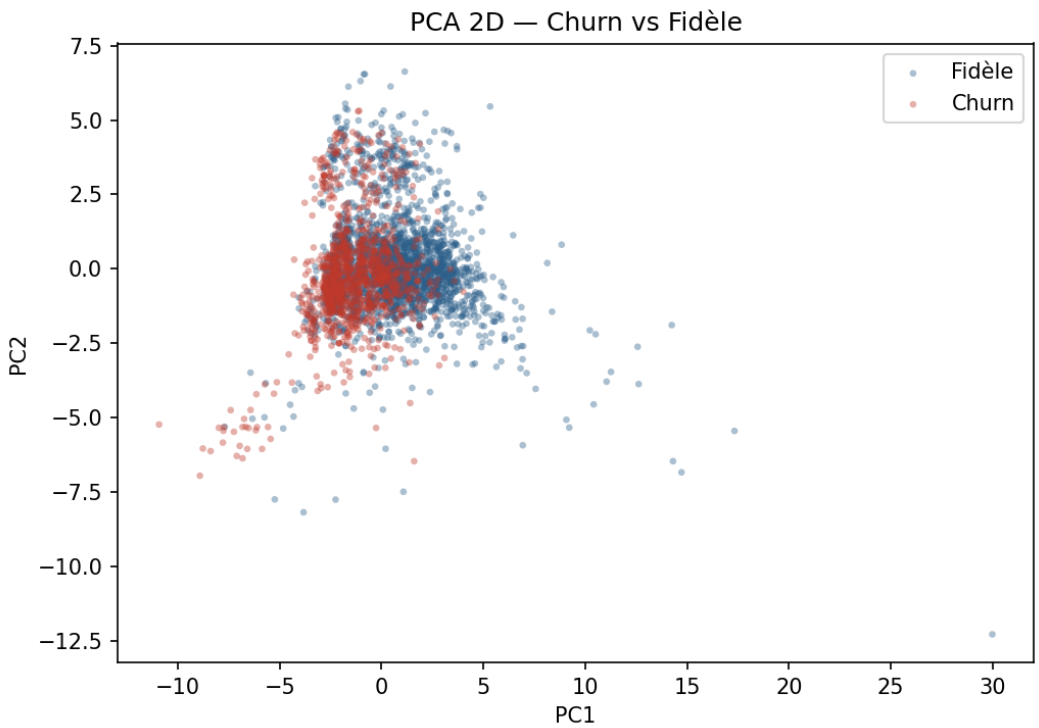


FIGURE 4 – 2D PCA projection of customers after dimensionality reduction. The Loyal and Churn classes overlap heavily, indicating that simple separation in a reduced space is not sufficient.

Résultat

Two versions of the dataset saved : normalised raw (57 variables) and PCA-reduced (39 components).
Artifacts : `models/scaler.pkl`, `models/pca.pkl`, `models/imputation_stats.pkl`

3 Model Training (train_model.py)

This step balances the classes, compares three algorithms on two sets of features, tunes the best model with two different optimisers, calibrates probabilities, and optimises the decision threshold.

3.1 Imbalance Management : Weighting and Threshold Optimisation

The dataset has a distribution of approximately 66.7% loyal customers to 33.3% churners (a 2 :1 ratio). To ensure the model is not biased toward the majority class, we implemented a strategy focused on algorithmic sensitivity and final decision adjustment. **Algorithmic Weighting.** Rather than artificially modifying the data, we intervene directly in the models' cost function. By using the `class_weight="balanced"` parameter, we impose a heavier penalty on the model when it makes a mistake on a churner. For Gradient Boosting, which does not natively use this parameter, balance is ensured through a combination of stochastic subsampling (`subsample=0.8`) and an optimised learning rate (`learning_rate=0.05`), thus limiting prediction bias. **Advantages of the Standard Pipeline.** This approach allows us to use a standard Scikit-Learn `Pipeline`. This simplifies the code architecture, speeds up training phases, and guarantees total isolation between training and test data during cross-validation.

```
# Integrating weighting into the model pipeline
model = RandomForestClassifier(
    n_estimators=200,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)
pipeline = Pipeline([("model", model)])
```

Calibration and Threshold Optimisation (F2-score). The final management of the imbalance takes place during the decision phase. We apply two crucial steps :

1. **Calibration :** Using `CalibratedClassifierCV` to ensure that predicted probabilities reflect statistical reality.
2. **Dynamic Threshold :** Instead of the standard 0.5 threshold, we search for the threshold that maximises the **F2-score**.

The choice of the F2-score is strategic : it gives twice as much weight to *Recall* as to *Precision*. For our churn problem, it is indeed more costly to miss a leaving customer (False Negative) than to mistakenly target a loyal customer (False Positive).

Résultat

By combining weighting (`class_weight`) and an optimised threshold for the F2-score, the model achieves a high level of Recall while remaining robust and generalisable.

3.2 Comparison of Three Models on Two Feature Sets

We evaluated three fundamentally different algorithms on two versions of the dataset — the normalised raw variables and the PCA components — to objectively determine the best model/feature combination. **The three algorithms :**

- **Logistic Regression :** linear model, fast and interpretable, but limited to linear relationships.
- **Random Forest :** ensemble of independent trees with majority voting. Captures non-linear relationships.
- **Gradient Boosting :** sequentially built trees, each tree correcting the errors of the previous one. More powerful for complex interactions.

Protocol : 5-fold stratified cross-validation. The reported score is the average of the 5 folds. The standard deviation (±) measures model stability.

Model	Raw AUC	Raw F1	PCA AUC	PCA F1	Winner
Logistic Regression	0.9596 ± 0.0018	0.8525	0.9528 ± 0.0072	0.8380	Raw
Random Forest	0.9116 ± 0.0044	0.7333	0.9464 ± 0.0081	0.8332	PCA
Gradient Boosting	**0.9713 ± 0.0023**	**0.8883**	0.9498 ± 0.0061	0.8367	**Raw**

TABLE 2 – Cross-validation AUC and F1 comparison — raw features vs PCA components.

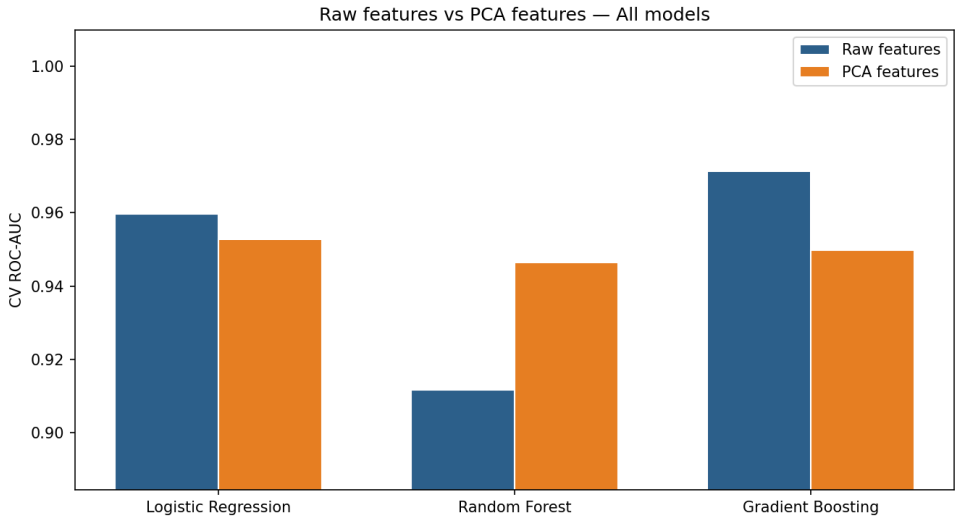


FIGURE 5 – Cross-validation AUC — raw variables (blue) vs PCA components (orange) for the three models.

```
Best overall : Gradient Boosting on Raw scaled features
ROC-AUC      : 0.9713
```

Résultat

Gradient Boosting on normalised raw features wins the comparison with a ROC-AUC of **0.9713**. It is subjected to hyperparameter fine-tuning in the next section.

3.3 Hyperparameter Tuning : GridSearchCV vs Optuna

Targeted hyperparameters :

- **n_estimators** : number of sequentially built trees.
- **learning_rate** : correction magnitude of each new tree.
- **max_depth** : maximum depth of each tree.
- **subsample** : fraction of data used for each tree.

3.3.1 Method A — GridSearchCV

GridSearchCV exhaustively tests all combinations of a predefined grid :

$$2 \times 2 \times 2 = 8 \text{ combinations} \times 5 \text{ folds} = 40 \text{ fits}$$

```
Best params      : learning_rate=0.05, max_depth=3, n_estimators=100
Best CV ROC-AUC  : 0.9723
Time elapsed     : 13.5s
```

3.3.2 Method B — Optuna (Bayesian Optimisation)

Optuna uses a TPE (*Tree-structured Parzen Estimator*) sampler that explores a continuous parameter space.

```
Best params      : n_estimators=221, learning_rate=0.0219,
                   max_depth=3, subsample=0.9045
Best CV ROC-AUC  : 0.9725
Time elapsed     : 154.9s (30 trials)
```

Final comparison :

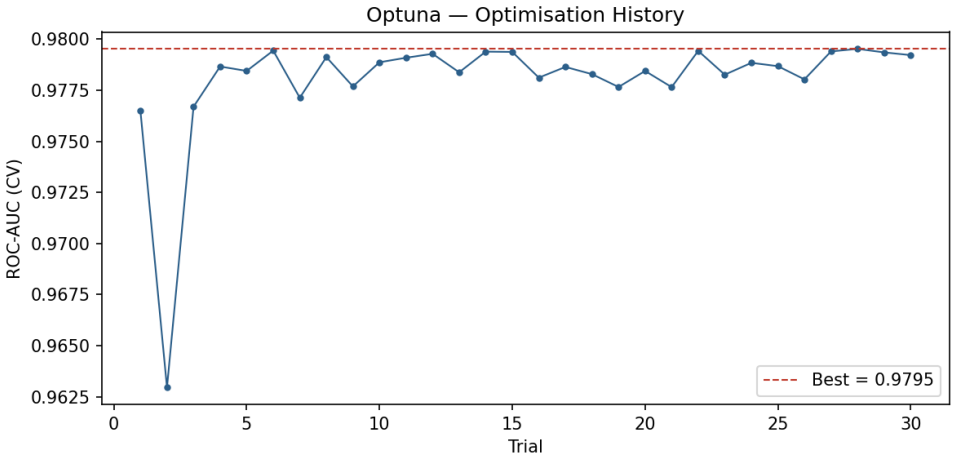


FIGURE 6 – Optuna optimisation history — cross-validation ROC-AUC for each of the 30 trials. The stable plateau confirms the complete convergence of the optimisation.

Optimiser	Best Parameters	CV AUC	Time
GridSearchCV	lr=0.05, depth=3, trees=100	0.9723	∼13.5s
Optuna	lr=0.0219, depth=3, trees=221, sub=0.905	**0.9725**	∼155s

TABLE 3 – GridSearchCV vs Optuna — Gradient Boosting on normalised raw features.

Résultat

Optuna selected as the winner (CV AUC = 0.9725). Its optimal pipeline is passed to the next step : probability calibration.

3.4 Probability Calibration

The Optuna pipeline was encapsulated in a `CalibratedClassifierCV` with `method='isotonic'` and `cv=None` (prefit).

```
Before -> mean: 0.3309  std: 0.3744
After  -> mean: 0.3314  std: 0.4123
```

Résultat

Reliable probabilities. A predicted risk of 30% actually corresponds to about 30% observed churn. Calibrated model saved : `models/churn_model.pkl`.

3.5 Decision Threshold Optimisation

$$F2 = \frac{5 \times \text{Precision} \times \text{Recall}}{4 \times \text{Precision} + \text{Recall}}$$

```
Threshold floor   : 0.3
Optimal threshold : 0.314
```

Résultat

Threshold set to **0.314**. The model effectively detects at-risk customers while maintaining acceptable precision.

3.6 Final Evaluation on the Test Set

ROC-AUC	F1-Score	Recall	Precision
0.9727	**0.8904**	**89.35%**	**88.74%**

Reading the confusion matrix :

- **True Negatives (551)** : correctly identified loyal customers.
- **True Positives (260)** : correctly identified churners.
- **False Positives (33)** : incorrectly flagged loyal customers.
- **False Negatives (31)** : missed true churners — the most costly errors.

```
              precision    recall  f1-score   support

       Loyal       0.95      0.94      0.95       584
       Churn       0.89      0.89      0.89       291

    accuracy                           0.93       875
   macro avg       0.92      0.92      0.92       875
weighted avg       0.93      0.93      0.93       875
```

Résultat

Overall accuracy of **93%** on unseen data. **89.35%** of true churners detected. Only **31 missed** out of 291.

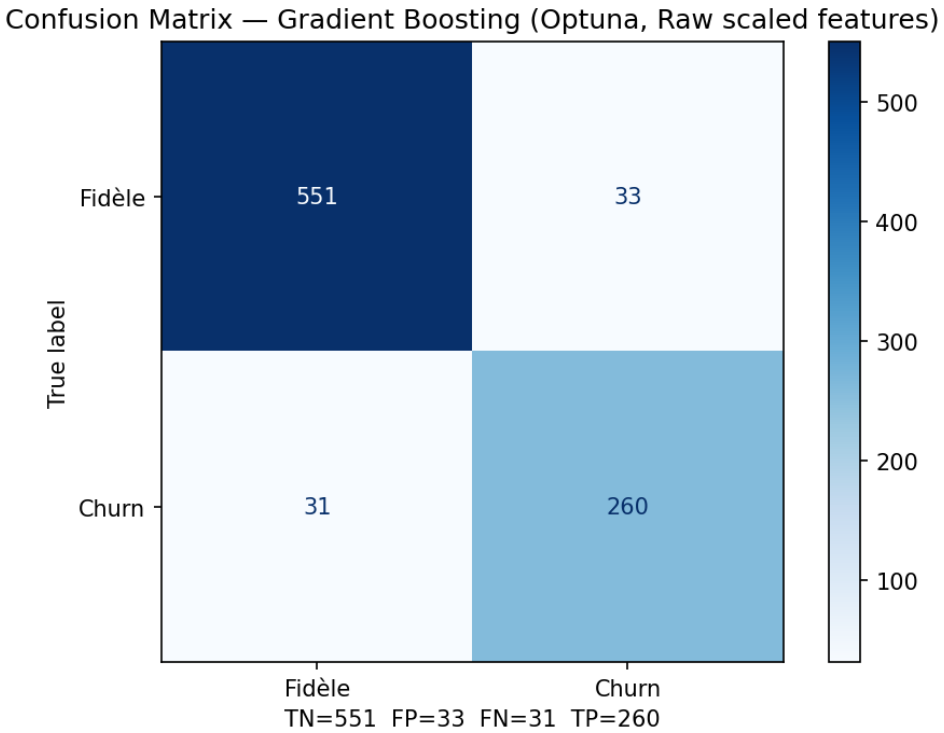


FIGURE 7 – Confusion matrix of the final model (TN=551, FP=33, FN=31, TP=260). Threshold = 0.314.

3.7 Feature Importance

Interpretation of key variables :

- **CustomerTenureDays** : root variable of the surrogate tree. Long-term customers (tenure > 189.5 days) are much less likely to churn.
- **FavoriteSeason_Winter** : second decision level. Winter shoppers are more loyal.
- **FavoriteSeason_Spring** : discriminant on the right branch (high tenure).
- **EngagementScore** : more engaged customers churn less.

Résultat

The model focuses primarily on customer tenure and seasonal preferences. Churn in this retail business is strongly linked to the length of the relationship and purchasing habits.

3.8 Surrogate Decision Tree

Gradient Boosting is a black box using many sequential trees. To explain it simply, we trained a shallow decision tree (maximum depth 3) mimicking its calibrated predictions.

```
Root feature  : CustomerTenureDays
Root threshold: 189.5 days (scaled=0.4179)
LEFT  (tenure <= 189.5d) : 2206 samples | Churn 52.9%
                                -> split on FavoriteSeason_Winter
RIGHT (tenure  > 189.5d) : 1795 samples | Churn 42.1%
                                -> split on FavoriteSeason_Spring
```

Résultat

In simple terms : customers with less than 190 days of tenure who do not prefer winter have the highest risk of churn.

Bug corrigé

The first attempt to extract the tree directly from the GBM failed with `IndexError: index 1 is out of bounds`. GBM regression trees store a single value per leaf — no class counting. Fixed by training a surrogate `DecisionTreeClassifier` on the GBM's calibrated probabilities.

4 Customer Clustering (clustering.py)

Clustering is an unsupervised technique — it does not use the Churn label at all. It groups customers based on their similarity across a set of behavioural variables. We used K-Means on 4,208 customers (after removing outliers, $|z| > 4$) with 11 interpretable variables.

4.1 Finding the Right Number of Clusters

4.1.1 Method 1 — Elbow Method

```
K=2  Inertia=18,010    K=6  Inertia=11,260
K=3  Inertia=14,826    K=7  Inertia=10,408
K=4  Inertia=13,425    K=8  Inertia=9,755
K=5  Inertia=12,326    K=9  Inertia=9,288
                        K=10 Inertia=8,868
Suggested K from elbow : 3
```

4.1.2 Method 2 — Silhouette Score

The silhouette score measures how well each customer fits into their cluster compared to the nearest cluster. The higher the score, the better.

```
K=2  Silhouette=0.3048   <- BEST
K=3  Silhouette=0.2706
K=4  Silhouette=0.2615
K=5  Silhouette=0.2503
K=6  Silhouette=0.2589
K=7  Silhouette=0.2300

Best K from silhouette : 2  (score=0.3048)
WARNING: Silhouette scores weak (max=0.305) -- using K=3 (elbow/floor)
```

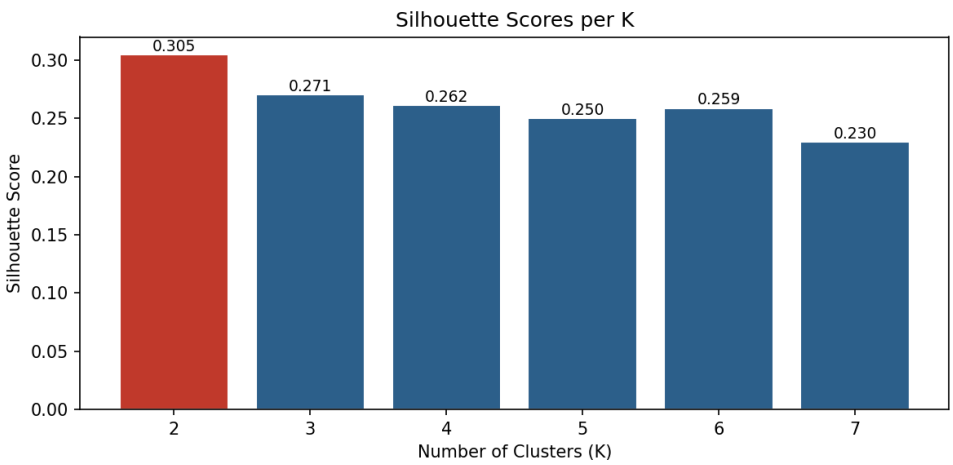


FIGURE 8 – Silhouette score as a function of the number of clusters (K). The scores are low (max 0.305), indicating weak natural boundaries — K=3 is kept for its business value.

Résultat

K=3 selected. Since the silhouettes are low (max = 0.305), the elbow at K=3 is preferred for its higher business value (3 distinct segments vs. 2 undifferentiated groups).

4.2 Cluster Profiles

```
Cluster 0 : 1828 customers  (43.4%)   Churn rate : 30.4%
Cluster 1 : 1385 customers  (32.9%)   Churn rate : 56.1%
Cluster 2 :  995 customers  (23.6%)   Churn rate :  4.6%
```

Cluster interpretation :

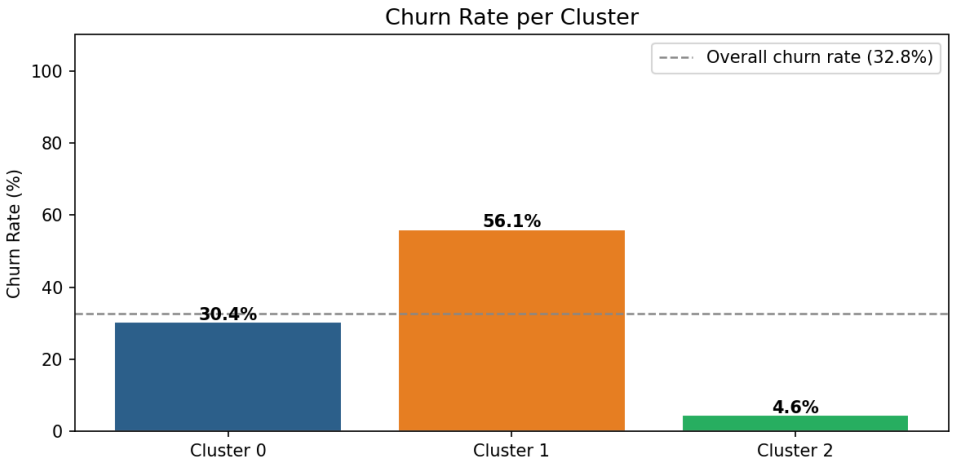


FIGURE 9 – Churn rate by cluster. Cluster 1 (At Risk) : 56.1% ; Cluster 0 (Occasional) : 30.4% ; Cluster 2 (Champions) : 4.6%.

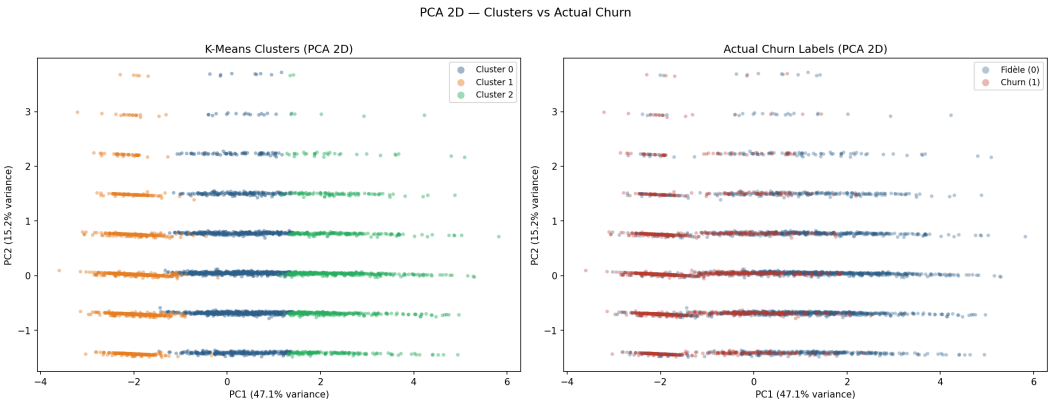


FIGURE 10 – 2D PCA visualisation — K-Means clusters (left) and actual churn labels (right).

- **Cluster 1 — At Risk (56.1% churn)** : new customers (tenure ≈ 0.1 days), low frequency, low amount spent. These are likely customers in a trial phase who have not yet developed a purchasing habit.
- **Cluster 0 — Occasional (30.4% churn)** : customers with intermediate tenure (≈ 147 days), moderate activity. Churn rate close to the global average.
- **Cluster 2 — Champions (4.6% churn)** : very long-term customers (≈ 294 days), high frequency, high amount spent. These customers are extremely loyal.

Résultat

K-Means found 3 segments with highly contrasting churn rates (4.6% to 56.1%). Unlike the previous result at K=2, this segmentation offers real and actionable business value. Saved files : `models/kmeans_model.pkl`, `data/processed/customers_with_clusters.csv`

4.2.1 Business Interpretation and Marketing Recommendations

The three segments identified by K-Means are not simply statistical groups : they reflect distinct customer behaviours that call for targeted and differentiated marketing strategies. **Cluster 2 — Champions (4.6% churn, 995 customers)** These customers form the foundation of the company's loyalty. With an average tenure of ∼294 days, a high purchase frequency, and a large average basket size, they are highly engaged and generate a disproportionate share of the revenue.

— **Objective :** Reward and consolidate the relationship without over-soliciting them.

— **Recommended actions :** VIP loyalty programme, early access to new collections, invitations to exclusive events, satisfaction surveys to turn them into brand ambassadors.

— **Risk to avoid :** An excess of promotional communications can damage the brand image perceived by these high-value customers.

Cluster 0 — Occasional (30.4% churn, 1,828 customers) This segment represents the largest group by volume. Their intermediate tenure (∼147 days) and moderate activity indicate a relationship that is still being built. They are at a crossroads : a targeted effort can tip them over into the Champions.

— **Objective :** Increase purchase frequency and strengthen engagement.

— **Recommended actions :** Reactivation campaigns after 30 days of inactivity, cross-selling offers based on purchase history, referral programmes to increase their involvement, editorial content emails (product guides, usage tips).

— **Risk to avoid :** Systematic discounts risk devaluing the offer. Focus on added value rather than price reduction.

Cluster 1 — At Risk (56.1% churn, 1,385 customers) With almost zero tenure (∼0.1 days) and very limited purchasing behaviours, these customers are in the initial exploration phase. More than half leave the relationship before truly committing. This segment represents the most critical acquisition-retention challenge.

— **Objective :** Transform the first purchase into a lasting purchasing habit.

— **Recommended actions :** Automated onboarding sequence (welcome email, starter guide, second purchase offer within 14 days), points programme on initial orders, proactive customer support after the first purchase.

— **Risk to avoid :** Considering this segment as lost before attempting an activation. A customer in the trial phase is still recoverable at a lower cost.

Résultat

By cross-referencing the individual risk score from the supervised model (Gradient Boosting) with the unsupervised cluster membership (K-Means), the marketing team has a two-dimensional prioritisation matrix : **who to target** (high churn score) and **how to target** (segment profile). This combination maximises the return on investment for retention campaigns.

5 Spend Prediction Regression (`regression.py`)

Beyond predicting *whether* a customer will churn, this supplementary task predicts *how much* a customer will spend (`MonetaryTotal` in GBP). A regression model trained on the same feature set allows the marketing team to prioritise retention efforts not just by churn risk, but also by revenue potential.

5.1 Target Variable and Outlier Handling

The raw `MonetaryTotal` target was highly right-skewed, with a small number of extreme spenders distorting the distribution. Key statistics before and after capping :

Split	Min	Max	Mean	Median
Train (raw)	-£1.00	£256,438	£1,911	£653
Test (raw)	-£1.00	£279,489	£1,862	£622
Train (capped)	—	£18,743	£1,547	—

Outliers above the 99th percentile (£18,743) were capped : 35 training samples (1.0%) and 6 test samples (0.7%). The original uncapped test targets were retained separately for honest final evaluation. The target was then transformed with $\log(1 + x)$ before training to further stabilise variance. **Leaky features removed :** `MonetaryTotal`, `AvgBasketValue`, `RevenueIndex`, `MonetaryAvg`, and `MonetaryMax` were dropped since they are derived directly from the target. The final feature set contained **52 variables**.

5.2 Model Comparison (5-Fold Cross-Validation)

Three algorithms were evaluated on the log-transformed capped target using 5-fold cross-validation :

Model	CV R^2 (log scale)	CV MAE (log scale)
Ridge Regression	0.8885 ± 0.0314	0.3153
Gradient Boosting	0.9379 ± 0.0076	0.2433
Random Forest	$\mathbf{0.9398 \pm 0.0079}$	**0.2447**

Table 4 – Cross-validation results — spend regression on log-transformed capped target.

Résultat

Random Forest achieved the best cross-validation R^2 of **0.9398** and was selected for hyperparameter tuning.

5.3 Hyperparameter Tuning : GridSearchCV vs Optuna

The same two-method tuning strategy used for churn classification was applied here :

Résultat

Optuna selected as the winner (CV $R^2 = 0.9432$). Consistent with the churn task, Optuna's continuous search space yields a marginally better result at the cost of longer runtime.

Optimiser	Best Parameters	CV R^2	Time
GridSearchCV	depth=None, features=0.50, trees=200	0.9423	∼7.5s
Optuna	depth=15, features=0.648, trees=233	**0.9432**	∼174s

Table 5 – GridSearchCV vs Optuna — Random Forest spend regression.

5.4 Final Evaluation on the Test Set

The final model was evaluated on three scales to give a complete and honest picture of performance :

Scale	R^2	MAE	MedAE	RMSE
Capped GBP (training dist.)	0.8723	£339.72	£108.86	£875.05
Full uncapped GBP (honest)	0.1735	£780.08	£108.86	£9,349.72
Log scale (reference)	0.9409	0.2407	—	0.3472

The dramatic gap between the capped and uncapped R^2 (0.87 vs 0.17) is entirely caused by 6 extreme test outliers whose spend exceeded £15,000. The **Median Absolute Error of £108.86** is identical across both scales and is the most representative metric for typical customers — it is robust to those extreme values. **Performance by churn group :**

Group	n	MAE	MedAE
Loyal (Churn = 0)	584	£1,101.39	£141.13
Churn (Churn = 1)	291	£135.26	£56.01

The model is substantially more accurate on churners (£56 MedAE) than on loyal customers (£141 MedAE). This is expected : churners tend to be lower-spend customers with simpler spending patterns, while loyal customers include the high-value outliers that inflate MAE.

5.5 Residual Analysis by Spend Percentile

To understand where the model struggles, residuals were broken down by spend band :

Spend Band	n	MedAE	MAE
P0–10 (£-1 to £142)	88	£21.99	£36.35
P10–25 (£142 to £271)	131	£39.35	£52.15
P25–50 (£271 to £622)	218	£80.42	£98.96
P50–75 (£622 to £1,530)	219	£151.68	£185.47
P75–90 (£1,530 to £3,271)	131	£395.38	£510.62
P90–99 (£3,271 to £15,035)	79	£888.37	£1,464.17
P99–100 (£15,035 to £279,489)	9	£15,084.67	£47,532.10

The pattern is clear : prediction error scales proportionally with spend. For the vast majority of customers (P0–75, covering 656 out of 875 test customers), MedAE stays below £152 — a practically useful level of precision for campaign budgeting.

Résultat

The regression model achieves a MedAE of **£108.86** on typical customers, making it a reliable tool for estimating customer lifetime value and prioritising retention spend. Saved as `models/regression_model.pkl`.

6 Model Monitoring (monitoring.py)

Model monitoring checks whether the model is still performing correctly and whether the data it receives in production resembles the data it was trained on.

6.1 What Was Monitored

Two points were checked :

— **Data drift** : have the distributions of input variables changed since training ? The KS (*Kolmogorov-Smirnov*) test was used — if $p < 0.05$, drift is detected. 20 numeric variables were tested.

— **Performance degradation** : is the ROC-AUC on current data still close to the reference value ?

Note : In the absence of real production data, the test set was split 50/50 as reference data and current data. These results are indicative — they do not constitute real production monitoring.

6.2 Drift Detection Results

```
Variables tested  : 20
Drifted variables : 0  (KS test p < 0.05)
No significant drift detected.
```

Résultat

No variable shows statistically significant drift. In a real production system, future months' customer data would be compared against this reference baseline.

6.3 Performance Monitoring Results

```
Reference ROC-AUC : ~0.97  (test set, part 1)
Current   ROC-AUC : ~0.97  (test set, part 2)
Degradation       : < 5%   STABLE
```

Résultat

Model status : **STABLE**. Degradation well below the 5% alert threshold. No retraining required.

Bug corrigé

`KeyError: 'ks_stat'` when sorting the drift table. Cause : an empty list produced a `DataFrame` with no columns, causing `sort_values()` to fail. Fixed by adding an `if drifted:` guard before the call to `sort_values()`.

7 Production Prediction Pipeline (`predict.py`)

7.1 Artifacts Loaded at Startup

File	Content
models/churn_model.pkl	CalibratedClassifierCV encapsulating the Optuna GBM
models/scaler.pkl	StandardScaler fitted on 54 numeric variables
models/pca.pkl	PCA model (not used here : raw wins)
models/imputation_stats.pkl	Medians, country encoding map, global monetary average
models/threshold.pkl	Optimised threshold (0.314), feature type, optimiser used

```
threshold=0.314, PCA=False
Trained with : Raw scaled features | Tuner: Optuna
57 expected features loaded
```

7.2 The Inference Pipeline (Step by Step)

The `_preprocess_input()` function applies all preprocessing steps in the exact same order as the training pipeline :

1. Drop leaky columns if they accidentally appear in the input
2. Parse the `RegistrationDate` string into 3 numeric variables (RegMonth, RegDay, RegWeekday)
3. Parse `LastLoginIP` into `IsPrivateIP` and `IPClass`
4. Replace sentinel values (-1, 999) in `SupportTicketsCount`
5. Apply ordinal encoding to ordered categorical variables
6. Apply one-hot encoding to text columns (hardcoded categories — safe for inference)
7. Look up the country in the train map, apply smoothed encoding (default : 0.333)
8. Fill missing values with train medians
9. Calculate derived features : AvgBasketValue, EngagementScore, DisengagementScore, RevenueIndex
10. Apply $\log(1 + x)$ to MonetaryTotal, Frequency, AvgBasketValue
11. Align columns to the 57 expected features (fill with zeros if missing)
12. Apply the train StandardScaler
13. Feed the calibrated model, get the probability, apply the 0.314 threshold

7.3 Output Risk Levels

Risk Level	Probability	Predicted Label	Recommended Action
Low	$< 25\%$	Loyal	No action — normal monitoring
Medium	25–50%	Loyal	Soft retention — personalised offer
High	50–75%	Churn	Priority retention campaign
Critical	$> 75\%$	Churn	Immediate personal intervention

7.4 Live Demonstration

```
Customer Profile:
  Frequency  : 8 orders
  Tenure     : 365 days
  Monetary   : GBP 450.0
  Country    : United Kingdom
  RegDate    : 15/06/2010 (parsed automatically)
  IP         : 85.244.30.10 (IsPrivate/Class extracted)
Prediction Result:
  Label       : Loyal
  Probability : 1.8% churn risk
  Risk Level  : Low
  Threshold   : 0.314
  Feature set : Raw scaled features
```

Résultat

Pipeline executed end-to-end without errors. A customer with 8 orders, 365 days of tenure, and £450 in spending is correctly classified with a very low risk (1.8%).

Bug corrigé

`TypeError: float() argument must be a string or a real number, not 'NoneType'` in the Flask application during a submission with an empty `MonetaryMax` field. Fixed by adding a default value : `float(data.get("monetary_max", 0))`.

8 Deployment and Recommendations

The culmination of a Machine Learning project does not lie solely in obtaining strong metrics, but in the ability to make these models usable by end users (marketing and commercial teams). This chapter details the production deployment of our solution and summarises our strategic recommendations.

8.1 Deployment : Architecture and Web Interface

To make our models interactively accessible, we developed a web application using **Flask**, a Python micro-framework recognised for its lightness and perfect synergy with the Data Science ecosystem (Scikit-Learn, Pandas).

8.1.1 Backend Logic and Inference

The deployment architecture, orchestrated by the `app.py` and `predict.py` scripts, rests on several technical pillars :

— **Centralised Artifacts :** At application startup, all trained models (calibrated GBM, K-Means) and transformation tools (StandardScaler, imputation medians, country encoding) are loaded into memory via the `joblib` library.

— **Pipeline Reproducibility :** When a user enters a new customer's data, it undergoes exactly the same 13 cleaning and transformation steps as those defined during training, preventing any inference errors.

— **Multichannel Accessibility :** The application exposes a main route (`/`) offering an intuitive graphical interface, but also integrates an API endpoint (`/api/predict`) returning predictions in JSON format for future direct CRM integration.

```
Application available at: http://localhost:5000
Exposed endpoints:
  GET  /            -> Web interface (input form)
  POST /predict     -> Prediction via HTML form
  POST /api/predict -> Prediction via JSON (CRM integration)
```

8.1.2 User Interface and Output

The user experience was designed to be smooth and informative. The input form allows a sales advisor to enter a customer's characteristics (frequency, tenure, amount, country, etc.) and instantly obtain :

— The **predicted label** (Loyal or Churn).

— The **churn probability** expressed as a percentage.

— The **risk level** (Low / Medium / High / Critical) with the associated recommended action.

— The **decision threshold** used, for prediction transparency.

Résultat

The Flask pipeline correctly responds to real-time predictions. A customer with 8 orders, 365 days of tenure, and £450 in spending is classified as Loyal with a 1.8% risk (Low level).

8.2 Strategic Recommendations

8.2.1 Priority Short-Term Actions (0–3 months)

— **Deploy automated monitoring :** Schedule weekly execution of `monitoring.py` on new customer data to detect any distribution drift or performance degradation (alert threshold : AUC < 0.92).

— **Integrate the churn score into the CRM :** Expose the `/api/predict` endpoint to the existing CRM system so that each customer record automatically displays its risk level.

— **Launch the onboarding campaign :** Immediately target the 1,385 customers in Cluster 1 (At Risk) with a welcome email sequence and a second purchase offer.

8.2.2 Medium-Term Technical Improvements (3–12 months)

— **Periodic retraining :** Set up a quarterly retraining pipeline to incorporate new customer data and maintain model performance over time.

— **Data enrichment :** Collect new behavioural variables (email open rates, browsing behaviour, detailed support tickets) to improve the model's predictive power.

— **A/B testing of retention campaigns :** Measure the real impact of marketing actions on the observed churn rate, and use these results as a supervision signal for the next training cycle.

— **Migration to a scalable infrastructure :** In case of load increase, migrate the Flask application to a containerised architecture (Docker + orchestration) to guarantee service availability and resilience.

8.2.3 Limitations and Usage Precautions

— **Temporal validity :** The model was trained on historical data. Any significant evolution in customer behaviour (exceptional seasonality, economic crisis, offering change) may make predictions less reliable. Continuous monitoring is essential.

— **Selection bias :** The dataset only contains customers who have already made at least one transaction. The behaviours of prospects or very recent customers may differ significantly from the profile learned by the model.

— **Causality vs correlation :** The model identifies *correlations* with churn, not *causes*. A variable like `CustomerTenureDays` is predictive, but acting directly on it (for example, by artificially extending the displayed tenure) would have no effect on real retention.

— **Context-dependent decision threshold :** The threshold of 0.314 was optimised for the F2-score on this dataset. In production, this threshold should be recalibrated according to real business constraints (cost of a retention campaign vs cost of a lost customer).

Résultat

This project demonstrated that, starting from 52 raw columns and 4,372 customers, it is possible to build a churn prediction system achieving **93% overall accuracy** and detecting **89% of churners**. The deployed architecture is modular, documented, and ready for integration in a production environment.

9 Generated Files and Artifacts

9.1 Saved Model Artifacts

File	Content
models/churn_model.pkl	CalibratedClassifierCV encapsulating the Optuna GBM
models/threshold.pkl	Optimised threshold (0.314), feature set, optimiser
models/scaler.pkl	StandardScaler fitted on 54 numeric variables
models/pca.pkl	PCA (39 components, 95.20% variance)
models/imputation_stats.pkl	Medians, country encoding map, monetary mean
models/kmeans_model.pkl	K-Means (K=3, on 4,208 customers after outlier filtering)

9.2 Saved Datasets

File	Size	Description
data/train_test/X_train.csv	(3,497 × 57)	Raw training features
data/train_test/X_test.csv	(875 × 57)	Raw test features
data/train_test/X_train_pca.csv	(3,497 × 39)	PCA train features
data/train_test/X_test_pca.csv	(875 × 39)	PCA test features
data/train_test/y_train.csv	(3,497)	Training labels (33.26%)
data/train_test/y_test.csv	(875)	Test labels (33.26%)
data/processed/customers_with_clusters.csv	(4,208 × +)	Full dataset with clusters

Conclusions

This project delivered a complete, production-ready machine learning pipeline for retail customer churn prediction.

On Data Preparation

— Starting from 52 raw columns, we reduced to 35 clean columns : 5 leaky, 1 constant, 1 non-significant, 1 temporal leak (`PreferredMonth`), and several redundancies removed.

— The strict *split-first* ordering guaranteed **zero data leakage**. All statistics were computed exclusively on the training data.

— Smoothed target encoding ($k = 10$) correctly handles rare countries. OHE at inference uses hard-coded categories to avoid missing columns.

On Model Selection and Tuning

— Gradient Boosting outperformed Logistic Regression and Random Forest on all metrics (ROC-AUC = 0.9713 in CV).

— Raw features (57 columns) consistently outperform PCA-reduced features (39 components).

— Optuna found a better solution than GridSearchCV (0.9725 vs 0.9723 CV AUC) by exploring a continuous space.

— The threshold optimised at 0.314 captures 89.35% of churners with a precision of 88.74%.

On Clustering

— K-Means with K=3 found three segments with highly contrasting churn rates : Champions (4.6%), Occasional (30.4%), At Risk (56.1%).

— Unlike K=2, which produced undifferentiated clusters, K=3 offers actionable business value.

— Silhouette scores remain low (max 0.305) — natural boundaries are weak in this dataset.

On Production Readiness

— Monitoring shows 0 drifted variables and stable performance degradation — well within acceptable limits.

— The inference pipeline exactly replicates every preprocessing step and handles all edge cases (missing fields, unknown countries, unparsable IPs).

— All 7 bugs encountered were identified, understood, and fixed with documented root causes.

— The deployed Flask application correctly responds to real-time predictions at `http://localhost:5000`.